

RAG CHEATSHEET

Zero to Hero: Core Concepts & Techniques

■ CONTENTS

- 1. RAG Fundamentals
- 2. Chunking Strategies
- 3. Embedding Principles
- 4. Vector Search & Indexing
- 5. Retrieval Techniques
- 6. Advanced Architectures
- 7. Query Transformation
- 8. Evaluation Framework
- 9. Production Patterns
- 10. Cost Optimization

Compiled by **Hesham Haroon**
AI Team Lead | 7+ Years Production Experience

1 RAG FUNDAMENTALS

What is RAG?

Retrieval-Augmented Generation combines LLMs with external knowledge retrieval. Instead of relying solely on parametric knowledge (what the model learned during training), RAG retrieves relevant context at inference time.

Why RAG?

Problem	How RAG Solves It
Hallucination	Grounds responses in retrieved evidence
Stale Knowledge	Retrieves from up-to-date sources
No Source Attribution	Can cite retrieved documents
Generic Responses	Provides domain-specific context
Expensive Retraining	Update knowledge without retraining

The RAG Pipeline

OFFLINE (Indexing): 1. **Ingest:** Load documents from various sources 2. **Chunk:** Split into retrieval units 3. **Embed:** Convert chunks to vectors 4. **Store:** Index in vector database **ONLINE (Query):** 1. **Embed Query:** Convert user query to vector 2. **Retrieve:** Find top-K similar chunks 3. **Rerank:** Re-score for relevance (optional) 4. **Augment:** Combine query + context 5. **Generate:** LLM produces grounded response

When to Use RAG vs Alternatives

Use RAG When	Use Fine-tuning When	Use Long Context When
Knowledge changes frequently	Need behavior/style change	Analyzing single document
Need source attribution	Domain vocabulary needed	Full context always required
Privacy requires separation	Static domain knowledge	Low query volume
Can't retrain frequently	Can afford training compute	Simple Q&A; on provided doc

Chunking is how you split documents into retrieval units. The right strategy depends on your document structure and query patterns.

Chunking Methods

Method	How It Works	Trade-offs
Fixed Size	Split every N tokens	Fast but breaks context
Recursive	Split by hierarchy (\n\n → \n → .)	Respects structure, most common
Semantic	Split when meaning changes	Coherent but slower
Sentence	Split at sentence boundaries	Complete thoughts, variable size
Document	Keep full doc as one chunk	Full context but size limits
Agentic	LLM decides boundaries	Best quality, expensive

Chunk Size Principles

Smaller chunks (256-512 tokens): • More precise retrieval (less noise) • Better for specific factual queries • Risk: may lack sufficient context
Larger chunks (1024-2048 tokens): • More context per chunk • Better for complex reasoning • Risk: more noise, may hit context limits
Research finding: Page-level chunking showed highest accuracy (0.648) in NVIDIA study across diverse document types.

Overlap Strategy

Why overlap? Prevents information loss at chunk boundaries. **How much?** Typically 10-20% of chunk size (50-100 tokens for 512 chunks).
Trade-off: More overlap = more storage + potential duplicate retrieval.

Advanced Chunking Patterns

Pattern	Description	When to Use
Parent-Child	Retrieve small, return large parent	Need surrounding context
Contextual	Prepend doc summary to each chunk	Chunks lack context alone
Proposition	Extract atomic facts as chunks	Fact-dense documents
Hierarchical	Multi-level (doc → section → para)	Need different granularities

■ **KEY INSIGHT:** Start with recursive chunking at 512 tokens with 50 token overlap. Iterate based on retrieval evaluation.

3 EMBEDDING PRINCIPLES

Embeddings convert text to dense vectors that capture semantic meaning. Similar texts have similar vectors.

How Embeddings Work

1. **Tokenization**: Text → tokens 2. **Encoding**: Tokens → transformer layers → hidden states 3. **Pooling**: Hidden states → single vector (mean, CLS, etc.) 4. **Normalization**: Vector → unit length (for cosine similarity)

Embedding Model Properties

Property	What It Means	Impact
Dimensions	Vector size (384-3072)	Higher = more nuance, more storage
Max Tokens	Input length limit (256-32K)	Longer = handle bigger chunks
Training Data	What model learned from	Domain match matters
Asymmetric	Query vs passage distinction	Some need different prefixes

Selection Criteria

• **Domain match**: General vs specialized (legal, medical, code) • **Language**: Monolingual vs multilingual support • **Context length**: Must handle your chunk sizes • **Latency/cost**: API vs local, batch size limits • **Benchmark scores**: MTEB for general, domain-specific evals

Embedding Best Practices

Practice	Why It Matters
Same model for index + query	Different models = incompatible vector spaces
Normalize vectors	Required for cosine similarity to work correctly
Batch embedding calls	Reduces latency and API costs (100-500 per batch)
Cache embeddings	Don't re-embed unchanged documents
Use query prefixes if required	Some models distinguish query vs passage

■■ **CRITICAL**: Never mix embedding models between indexing and querying. Vectors from different models exist in incompatible spaces.

Distance Metrics

Metric	Formula	When to Use
Cosine Similarity	$\text{dot}(A,B) / (\ A\ \times \ B\)$	Normalized vectors, text (most common)
Euclidean (L2)	$\sqrt{\sum (a_i - b_i)^2}$	Absolute distances matter, images
Dot Product	$\sum (a_i \times b_i)$	Pre-normalized vectors, faster
Manhattan (L1)	$\sum a_i - b_i $	Sparse vectors, some domains

Index Types

Index	How It Works	Trade-off
Flat (Exact)	Compare to every vector	100% recall, O(n) slow
IVF	Cluster vectors, search clusters	Fast, loses some recall
HNSW	Graph-based navigation	Very fast, high memory
PQ	Compress vectors	Low memory, loses precision
IVF_PQ	Cluster + compress	Balanced for large scale

Index Tuning Parameters

HNSW: • M (connections per node): Higher = better recall, more memory • ef_construction: Higher = better index quality, slower build • ef_search: Higher = better recall at query time, slower **IVF:** • nlist (clusters): $\sqrt{n}$ to $4\sqrt{n}$ where n = vector count • nprobe (clusters to search): Higher = better recall, slower

Metadata Filtering

Pre-filtering: Filter THEN search vectors • Pros: Guaranteed to respect filters • Cons: May search small subset, less efficient **Post-filtering:** Search vectors THEN filter • Pros: Efficient vector search • Cons: May return fewer results than requested **Best practice:** Pre-filter for critical filters, post-filter for nice-to-haves.

Retrieval Methods

Method	Mechanism	Strengths	Weaknesses
Dense	Vector similarity	Semantic understanding	Misses exact terms
Sparse (BM25)	Term frequency	Exact matches, keywords	No semantics
Hybrid	Combine both	Best of both worlds	More complex

■■ **CRITICAL:** Pure vector search fails in production. 'Error 503' may match 'Error 404' semantically. Hybrid search is mandatory for production.

Hybrid Fusion Methods

Reciprocal Rank Fusion (RRF): $\text{score}(d) = \sum 1/(k + \text{rank}_i(d))$ where $k = 60$ (constant) • Combines rankings without needing score normalization • Works well when score distributions differ **Weighted Sum:** $\text{score} = \alpha \times \text{normalized_vector} + (1-\alpha) \times \text{normalized_bm25}$ • Requires score normalization (min-max or z-score) • α typically 0.5-0.7 for balanced results

Reranking

What: Re-score top-K results using a more powerful model **Why:** Cross-encoders see query + doc together (not separate embeddings) **How:** Retrieve top-100 with fast retrieval → rerank to top-10 **Cross-encoder vs Bi-encoder:** • Bi-encoder: Encode query and doc separately (fast, scalable) • Cross-encoder: Encode query+doc together (slow, accurate)

Maximal Marginal Relevance (MMR)

Purpose: Balance relevance with diversity (reduce redundancy) **Formula:** $\text{MMR} = \lambda \times \text{sim}(q, d) - (1-\lambda) \times \max(\text{sim}(d, d_{\text{selected}}))$ λ : 1.0 = pure relevance, 0.0 = pure diversity, typical = 0.5-0.7

Naive RAG: Simple retrieve → generate pipeline. Good for POC, fails at scale.

Advanced RAG: Pre-retrieval (query rewriting) + post-retrieval (reranking, compression) optimizations.

Modular RAG: Decomposed into swappable modules: retriever, reranker, generator, etc.

Self-RAG: Model self-reflects: Should I retrieve? Is it relevant? Is answer grounded? Requires special training.

Corrective RAG (CRAG): Evaluates retrieval: Correct → use | Ambiguous → refine | Wrong → fallback to web search.

RAPTOR: Builds hierarchical tree of summaries. Query can match at any abstraction level.

GraphRAG: Extracts entity-relationship graph. Enables multi-hop reasoning across entities.

Agentic RAG: Agent decides what/when/whether to retrieve. Can use multiple tools (RAG + SQL + API).

Architecture Selection

Requirement	Architecture
Simple Q&A, single source	Advanced RAG
Need to validate retrieval quality	CRAG
Questions at different abstraction levels	RAPTOR
Entity relationships, multi-hop	GraphRAG
Dynamic tool selection	Agentic RAG
Highest accuracy, can train model	Self-RAG

7 QUERY TRANSFORMATION

Transform user queries to improve retrieval. The query as-typed often doesn't match how information is stored.

Transformation Techniques

Technique	How It Works	When to Use
Query Rewriting	LLM rewrites for clarity/specificity	Vague or colloquial queries
Query Expansion	Add synonyms and related terms	Vocabulary mismatch
Multi-Query	Generate 3-5 query variations	Ambiguous queries (highest ROI)
HyDE	Generate hypothetical answer, embed that	Abstract questions
Step-back	Generate broader question first	Too specific queries
Decomposition	Break into sub-queries	Multi-part questions

■ **KEY INSIGHT:** Multi-query generation is the highest ROI improvement. LLM generates semantic + keyword variations, covering more search surface.

Query Routing

Not every query should go to RAG: • **Retrieval:** Factual questions answerable from your docs • **Generation:** Creative, summarization, analysis tasks • **Clarification:** Ambiguous → ask follow-up question • **Rejection:** Out of scope → decline politely **Implementation:** Classifier or LLM-based router before RAG pipeline.

Retrieval Metrics

Metric	Formula	Target
Recall@K	$\text{relevant_in_top_k} / \text{total_relevant}$	> 0.8
Precision@K	$\text{relevant_in_top_k} / K$	> 0.6
MRR	$\text{mean}(1 / \text{first_relevant_position})$	> 0.7
NDCG@K	$\text{DCG} / \text{ideal_DCG} \text{ (position-aware)}$	> 0.75
Hit Rate	$\text{queries_with_any_relevant} / \text{total_queries}$	> 0.9

Generation Metrics (RAGAS)

Metric	What It Measures	Target
Faithfulness	Is answer grounded in context?	> 0.9
Answer Relevancy	Does answer address query?	> 0.85
Context Precision	Is retrieved context relevant?	> 0.7
Context Recall	Is all needed info retrieved?	> 0.8
Answer Correctness	Does answer match ground truth?	> 0.7

Evaluation Strategy

1. **Create Test Set:** 200-500 queries with ground truth answers + relevant docs 2. **Retrieval Eval:** Recall@10, MRR, Hit Rate (don't skip this!) 3. **Generation Eval:** RAGAS metrics (Faithfulness most important) 4. **LLM-as-Judge:** GPT-4/Claude rates quality 1-5 (cheaper than human) 5. **Human Eval:** Sample 50-100 for calibration 6. **Production:** Continuous monitoring, alert on regression

■■ **CRITICAL:** 90% of RAG failures are due to inadequate evaluation, not architecture. Build eval from day one.

✓ Production Essentials

- ✓ Hybrid search from day one (vector + keyword)
- ✓ Add reranking for precision
- ✓ Implement semantic caching
- ✓ Version your embeddings and index
- ✓ Log queries, retrievals, and responses
- ✓ Monitor retrieval quality continuously
- ✓ A/B test changes before full rollout
- ✓ Implement graceful fallbacks

✗ Production Anti-Patterns

- ✗ Vector-only search (misses exact terms)
- ✗ Skipping evaluation ('seems to work')
- ✗ Stuffing maximum context (more ≠ better)
- ✗ Mixing embedding models
- ✗ Ignoring latency until crisis
- ✗ Deploying without fallback

Debugging Flowchart

Bad Answer? → Is relevant content in retrieved chunks? ■■ **NO**: Retrieval problem → check embedding, chunking, query ■■ **YES**: Generation problem → check prompt, context order, LLM **Low Recall**: Expand queries, add hybrid search, lower threshold **Low Precision**: Add reranking, tighten threshold, improve chunking **Hallucination**: Stricter prompt, require citations, use CRAG pattern

Cost Reduction Hierarchy

Technique	Savings	How It Works
Semantic Caching	50-70%	Cache similar queries (threshold 0.85-0.95)
Model Routing	40-60%	Small model for simple, large for complex
Prompt Caching	30-50%	Reuse processed system prompts
Context Compression	20-40%	Extract only relevant portions
Batch Processing	20-30%	Group non-urgent requests
Embedding Caching	10-20%	Don't re-embed unchanged docs

Semantic Caching Architecture

Layer 1: Exact match cache (fast, simple) **Layer 2:** Semantic similarity cache (embed query, search cache, threshold > 0.9) **Layer 3:** Cross-encoder validation (filter false positives before returning) **Key:** Balance cache hit rate vs false positive rate. Start conservative (0.95), tune down.

Model Routing Pattern

Tier 1 (70% queries): Simple model for straightforward Q&A; **Tier 2 (25% queries):** Medium model for moderate complexity **Tier 3 (5% queries):** Best model for complex reasoning **Router:** Trained classifier or rule-based (query length, keywords, complexity signals)

Production RAG Evolution Path

Stage 1: Naive RAG (vector only) → POC **Stage 2:** + Hybrid Search → +8-15% accuracy **Stage 3:** + Reranking → +5-10% precision **Stage 4:** + Multi-Query → +10-15% coverage **Stage 5:** + Semantic Caching → 50%+ cost reduction **Stage 6:** + Model Routing → 30-40% additional savings